World Bank Indicators API

Dataset-Specific Laboratory Workbook

Data Acquisition & Manipulation with Pandas

GROUP Group C • Source: World Bank Indicators API v2 — JSON

How to use this workbook

You support a development-policy unit comparing population and economic indicators across selected countries and years. Here, acquisition itself is programmable: parameters, pagination, HTTP status and JSON normalization matter.

This workbook assumes Python essentials and notebook use. Its focus is not Python syntax revision; it is disciplined Pandas reasoning: acquire → inspect → diagnose → select/filter → clean/transform → group/aggregate → merge/reconcile → export.

ROW GRAIN For a single indicator call, one observation is typically country × indicator × period.

KEYS & IDENTITY country.id / countryiso3code, indicator.id and date identify observations. API metadata reports pages and totals.

Learning outcomes

- Acquire the dataset using the source-appropriate Pandas/API/database syntax.
- Diagnose structure, dtypes, missingness, duplicates, ranges, categories and memory implications before cleaning.
- Use loc/iloc, boolean masks, query, isin, between, sort_values and column selection intentionally.
- Clean and transform without overwriting raw data or silently changing analytical meaning.
- Use groupby, agg, named aggregation, value_counts, pivot/crosstab/transform where justified.
- Join/merge/concat with explicit key/cardinality reasoning and post-operation validation.
- Export reproducible outputs and explain why each transformation was necessary.

Acquisition starter

```
import requests
import pandas as pd

base = "https://api.worldbank.org/v2/country/NGA;GHA;KEN/indicator/SP.POP.TOTL"
params = {"format":"json", "date":"2015:2024", "per_page":1000}
r = requests.get(base, params=params, timeout=30)
r.raise_for_status()
payload = r.json()

metadata, records = payload
print(metadata)
pop = pd.json_normalize(records)
pop.head()
```

Core Pandas syntax map

Capability	Methods / properties
Inspect	shape, head(), tail(), sample(), columns, dtypes, info(), memory_usage(), describe(), nunique(), value_counts(), isna(), duplicated()
Select	df[col], df[[...]], loc[rows, cols], iloc[rows, cols], filter(), select_dtypes()
Filter	boolean masks with & | ~, isin(), between(), str methods, query(), notna()/isna()
Clean	copy(), dropna(), fillna(), drop_duplicates(), astype(), to_numeric(), to_datetime(), str.strip(), replace(), map(), rename()
Transform	assign(), where(), mask(), cut(), qcut(), apply()/map() when justified; prefer vectorization
Group	groupby(), size(), count(), sum(), mean(), median(), min(), max(), nunique(), agg(), transform(), pivot_table(), crosstab()
Combine	merge(how=..., on=..., validate=..., indicator=True), join(), concat(), suffixes
Validate/export	assert, equals(), compare counts/totals, to_csv(index=False), to_excel(index=False), to_parquet()

Case Study 1: API acquisition & pagination

REAL-LIFE QUESTION How do you prove you fetched the complete result rather than only page 1?

WHY THIS MATTERS A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

1. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
2. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
3. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
4. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
5. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
6. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

Guided implementation

```
meta, rows = payload
print(meta["page"], meta["pages"], meta["per_page"], meta["total"])

frames=[]
for page in range(1, int(meta["pages"])+1):
    p=params | {"page":page}
    rr=requests.get(base, params=p, timeout=30)
    rr.raise_for_status()
    m, data=rr.json()
    frames.append(pd.json_normalize(data))
pop_all=pd.concat(frames, ignore_index=True)
assert len(pop_all) == int(meta["total"])
```

Stop & Predict

- Which object(s) will change shape after this code?

- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

Checkpoint evidence

Evidence	Your result	Interpretation
Shape / row count	_____	_____
Missing / duplicate check	_____	_____
Key or total reconciliation	_____	_____

Practice ladder

Basic — Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

Intermediate — Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

Challenge — Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

Debug — Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

Case Study 2: Inspect/select normalized JSON

REAL-LIFE QUESTION Which fields are analytically useful after json_normalize?

WHY THIS MATTERS A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

7. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
8. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
9. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
10. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
11. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
12. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

Guided implementation

```
pop_all.shape
pop_all.info()
pop_all.columns.tolist()
pop_all.isna().sum()
pop = pop_all.loc[:, ["country.id","country.value","countryiso3code",
                      "indicator.id","indicator.value","date","value","unit","obs_status"]].copy()
pop["year"] = pd.to_numeric(pop["date"], errors="coerce").astype("Int64")
```

Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

Checkpoint evidence

Evidence	Your result	Interpretation
Shape / row count	_____	_____
Missing / duplicate check	_____	_____
Key or total reconciliation	_____	_____

Practice ladder

Basic — Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

Intermediate — Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

Challenge — Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

Debug — Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

Case Study 3: Clean & validate API observations

REAL-LIFE QUESTION Should a missing indicator value become zero?

WHY THIS MATTERS A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

13. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
14. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
15. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
16. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
17. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
18. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

Guided implementation

```
pop["value"] = pd.to_numeric(pop["value"], errors="coerce")
pop.duplicated(["countryiso3code","indicator.id","year"]).sum()
missing = pop.loc[pop["value"].isna()].copy()
complete = pop.loc[pop["value"].notna()].copy()
# Missing means unavailable/not reported; zero is a substantive observed value.
```

Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

Checkpoint evidence

Evidence	Your result	Interpretation
Shape / row count	_____	_____
Missing / duplicate check	_____	_____

Key or total reconciliation	_____	_____

Practice ladder

Basic — Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

Intermediate — Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

Challenge — Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

Debug — Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

Case Study 4: Group & aggregate coverage

REAL-LIFE QUESTION Which country has the best data coverage in the requested period?

WHY THIS MATTERS A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

19. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
20. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
21. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
22. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
23. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
24. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

Guided implementation

```
coverage = (
    pop.groupby("countryiso3code", as_index=False)
       .agg(years_requested=("year","nunique"),
            observations=("value","count"),
            missing=("value", lambda s: s.isna().sum()),
            min_value=("value","min"),
            max_value=("value","max"))
)
coverage
```

Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

Checkpoint evidence

Evidence	Your result	Interpretation
Shape / row count	_____	_____
Missing / duplicate check	_____	_____
Key or total reconciliation	_____	_____

Practice ladder

Basic — Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

Intermediate — Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

Challenge — Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

Debug — Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

Case Study 5: Merge two indicators

REAL-LIFE QUESTION Can you fetch GDP and population separately, then merge by country-year to compute GDP per person?

WHY THIS MATTERS A Pandas method is justified by the analytical question, the row grain, data quality, and the risk of changing meaning—not by habit.

25. **1. State the grain before coding:** Write one sentence defining what one row represents. Identify candidate keys and whether they should be unique.
26. **2. Inspect before transforming:** Run the inspection code. Record shape, dtypes, missingness, duplicates, ranges/categories, and suspicious values.
27. **3. Predict the effect:** Before running the transformation, predict whether row count, column count, null count, or grain should change.
28. **4. Execute the Pandas operation:** Run the code below one logical block at a time. Do not run the whole notebook blindly.
29. **5. Validate:** Compare before/after counts, key uniqueness, totals or category counts. A successful cell is not evidence that the result is correct.
30. **6. Interpret:** Write 2–4 sentences: what changed, why it was justified, what remains uncertain, and what you would tell the business user.

Guided implementation

```
def fetch_indicator(code):
    url=f"https://api.worldbank.org/v2/country/NGA;GHA;KEN/indicator/{code}"
    rr=requests.get(url, params={"format":"json","date":"2015:2024","per_page":1000}, timeout=30)
    rr.raise_for_status()
    _, data=rr.json()
    x=pd.json_normalize(data)
    x["year"]=pd.to_numeric(x["date"], errors="coerce").astype("Int64")
    return x[["countryiso3code","year","value"]].rename(columns={"value":code})

pop=fetch_indicator("SP.POP.TOTL")
gdp=fetch_indicator("NY.GDP.MKTP.CD")
panel=pop.merge(gdp, on=["countryiso3code","year"], how="inner",
                validate="one_to_one", indicator=True)
panel["gdp_per_person_calc"]=panel["NY.GDP.MKTP.CD"]/panel["SP.POP.TOTL"]
```

Stop & Predict

- Which object(s) will change shape after this code?
- Could this operation silently remove or duplicate valid records? How will you detect that?
- What alternative Pandas method could solve part of this task, and why is the chosen one preferable here?

Checkpoint evidence

Evidence	Your result	Interpretation
Shape / row count	_____	_____
Missing / duplicate check	_____	_____
Key or total reconciliation	_____	_____

Practice ladder

Basic — Modify one filter/selection condition and return only the columns needed to answer a narrower business question.

Intermediate — Produce a second summary using at least two of: query(), isin(), between(), sort_values(), value_counts(), agg(), transform(), pivot_table(). Explain the grain.

Challenge — Design a validation test with assert or explicit before/after reconciliation that would catch a plausible silent error.

Debug — Create or identify one wrong approach (chained assignment, == NaN, and/or, unvalidated merge, blanket dropna, or iterrows) and repair it.

Final Integrated Case

Management gives you the raw source and asks for a decision-ready table. Build an end-to-end notebook without copying the guided solution blocks. Your notebook must contain:

- source/provenance note and acquisition code
- grain and key statement
- dataset-health table
- documented filtering population
- cleaning decision log
- at least two derived features where meaningful
- at least two grouped summaries
- one merge/concat/reconciliation task appropriate to this dataset
- three validation assertions/checks
- clean export with index=False where CSV/Excel is used
- 200–300 word management note stating findings, exclusions, assumptions and limitations

Common pitfalls — non-negotiable

Pitfall	Required response
and / or in Series filters	Use & / | with each condition in parentheses.
Chained assignment	Use .loc[...] = ...; do not use df[df['a']>5]['b']=0.
Comparing NaN with ==	Use .isna() / .notna().
Modifying a slice	Use .copy() when creating a working subset.
Ignoring dtypes	Run info() early; IDs often should not be numeric measures.
Blind dropna/fillna	First explain what missing means in that field and question.
iterrows on large data	Prefer vectorized expressions/groupby/merge.
Unvalidated merge	Check key uniqueness, use validate= and indicator=True, reconcile rows/totals.
Overwriting raw files	Keep raw immutable; write clean/derived files separately.
CSV index leakage	Use to_csv(..., index=False) unless the index is intentionally part of the data.

Oral defence prompts

31. What does one row represent?
32. Which columns are identifiers rather than measures?
33. Which missing values did you retain, drop or impute—and why?
34. Show one operation that changed row count. Prove the change was intended.
35. What was the cardinality of your merge? How did you verify it?
36. Which metric would be wrong if you counted rows instead of unique business entities?
37. What would you change if this dataset had 50 million rows?
38. Which part belongs in SQL/API/source filtering rather than Pandas?
39. What conclusion can you NOT make from this dataset?

END OF DATASET LAB • Preserve raw data • Explain every decision • Validate every join